

# Foundations of Machine Learning

## Assignment 2 Report

CS25S023

October 26, 2025

### 1. Introduction

This assignment focuses on two fundamental machine-learning problems:

1. **Question 1: Mixture Models for Binary Data** Implement and compare Expectation–Maximization (EM) algorithms for mixture distributions — specifically a *Mixture of Bernoulli* and a *Mixture of Gaussian* model. The goal is to understand how EM behaves under correct and misspecified likelihood assumptions.
2. **Question 2: Linear Regression and Regularization** Implement analytical and iterative solutions for least-squares regression. Compare analytical, gradient-descent, stochastic-gradient-descent, and ridge-regularized variants, and study convergence and generalization through validation and test errors.

### Libraries Used

The following Python libraries were utilized for data preprocessing, dimensionality reduction, clustering, visualization, and performance evaluation:

- **Numpy:** For numerical computations and array manipulations.
- **Pandas:** For loading and processing datasets.
- **Matplotlib:** For data visualization and cluster plotting.

---

### Overview

---

This report presents a comprehensive analysis and implementation of the two core problems in Assignment 2 of *Foundations of Machine Learning*. It centers on probabilistic modeling and regression, combining theoretical derivations with empirical validation and performance evaluation. The first part investigates mixture modeling for high-dimensional binary data, deriving and implementing the EM algorithm for a product–Bernoulli mixture, and comparing its behavior with Gaussian mixtures and K-means clustering. The second part examines linear regression using analytical, gradient-based, and ridge-regularized methods, emphasizing convergence characteristics and generalization performance.

All algorithms are implemented from first principles in Python and are supported by clear visualizations and quantitative assessments. Notation follows standard conventions: bold lowercase letters (e.g.,  $\mathbf{x}$ ) represent vectors, bold uppercase letters (e.g.,  $\mathbf{X}$ ) denote matrices, and scalars are written in normal font. Mathematical formulations are complemented by empirical plots to ensure coherence between theory and practice throughout the report.

*Keywords:* mixture models, Expectation–Maximization, product Bernoulli, Gaussian mixture, K-means clustering, least squares, gradient descent, stochastic gradient descent, ridge regression.

## 2. Question 1: Mixture Models

### Problem Statement

You are given a dataset consisting of 400 data points in  $\{0,1\}^{50}$  generated from a mixture of some unknown distribution (provided in `A2Q1.csv`). Each data point represents a flattened version of a binary  $10 \times 5$  matrix. The goal is to identify the appropriate probabilistic mixture model that could have generated the data, derive and implement the corresponding EM algorithm, and analyze its performance.

The question is divided into the following parts:

- Part 1.** (a) Determine which probabilistic mixture could have generated this data (it is not a Gaussian mixture).  
 (b) Derive the EM algorithm for the chosen mixture and show all calculations.  
 (c) Implement the algorithm for  $K = 4$  mixture components.  
 (d) Plot the average log-likelihood (over 100 random initializations) versus iterations.
- Part 2.** (a) Assume the same data were generated from a mixture of Gaussians with  $K = 4$ . Implement EM and plot the average log-likelihood versus iterations.  
 (b) Compare the results with Part 1 and discuss insights.
- Part 3.** Run K-means with  $K = 4$  and plot the objective as a function of iterations.
- Part 4.** Compare all three algorithms and comment on which is most suitable for this dataset.

### 2.2 Identifying the Underlying Probabilistic Mixture

Based on the dataset characteristics, the data are best modelled as a **finite mixture of multivariate Bernoulli (product-Bernoulli) distributions**. Each component assumes feature-wise independence:

$$p(x_i | \theta_k) = \prod_{d=1}^D \theta_{kd}^{x_{id}} (1 - \theta_{kd})^{1-x_{id}},$$

where  $\theta_{kd} \in (0, 1)$  is the Bernoulli parameter for feature  $d$  in component  $k$ .

### 2.3 EM Derivation for a Mixture of Product Bernoullis

**Model formulation.** For  $x \in \{0, 1\}^d$  (here  $d = 50$ ), assume

$$p(x | \Theta) = \sum_{k=1}^K \pi_k \prod_{j=1}^d (p_j^{(k)})^{x_j} (1 - p_j^{(k)})^{1-x_j}, \quad (1)$$

where  $\Theta = \{\pi_k, p_j^{(k)}\}$ ,  $\pi_k \geq 0$ ,  $\sum_{k=1}^K \pi_k = 1$ , and  $p_j^{(k)} \in (0, 1)$ .

**Complete-data log-likelihood.** Let  $z^{(n)} \in \{1, \dots, K\}$  denote the latent component label for sample  $n$ . Then the complete-data log-likelihood is

$$\log p(X, Z | \Theta) = \sum_{n=1}^N \sum_{k=1}^K \mathbf{1}\{z^{(n)} = k\} \left[ \log \pi_k + \sum_{j=1}^d (x_j^{(n)} \log p_j^{(k)} + (1 - x_j^{(n)}) \log(1 - p_j^{(k)})) \right]. \quad (2)$$

#### E-step

In the expectation step, we compute the *responsibilities* (posterior probabilities of component membership):

$$r_{nk} = P(z^{(n)} = k | x^{(n)}, \Theta) = \frac{\pi_k \prod_{j=1}^d (p_j^{(k)})^{x_j^{(n)}} (1 - p_j^{(k)})^{1-x_j^{(n)}}}{\sum_{\ell=1}^K \pi_\ell \prod_{j=1}^d (p_j^{(\ell)})^{x_j^{(n)}} (1 - p_j^{(\ell)})^{1-x_j^{(n)}}}. \quad (3)$$

The effective cluster size for each component is

$$N_k = \sum_{n=1}^N r_{nk}. \quad (4)$$

### M-step

In the maximization step, we update parameters to maximize the expected complete-data log-likelihood  $Q(\Theta) = \mathbb{E}_{Z|X, \Theta^{\text{old}}} [\log p(X, Z | \Theta)]$ .

**Update for mixing weights:**

$$\pi_k \leftarrow \frac{N_k}{N}. \quad (5)$$

**Update for Bernoulli parameters:**

$$p_j^{(k)} \leftarrow \frac{\sum_{n=1}^N r_{nk} x_j^{(n)}}{N_k}, \quad j = 1, \dots, d, \quad k = 1, \dots, K. \quad (6)$$

To ensure numerical stability, parameters are clipped to  $p_j^{(k)} \in [\varepsilon, 1 - \varepsilon]$  (typically  $\varepsilon = 10^{-6}$ ) to avoid log 0 issues.

### Observed-data log-likelihood

At each iteration, the observed-data log-likelihood is evaluated as

$$\mathcal{L}(\Theta) = \sum_{n=1}^N \log \left( \sum_{k=1}^K \pi_k \prod_{j=1}^d (p_j^{(k)})^{x_j^{(n)}} (1 - p_j^{(k)})^{1-x_j^{(n)}} \right). \quad (7)$$

For numerical stability, the computation uses the *log-sum-exp* trick. EM guarantees that  $\mathcal{L}(\Theta)$  is non-decreasing across iterations.

**Convergence.** Monitor the log-likelihood at each iteration; EM guarantees non-decreasing  $\mathcal{L}$ .

#### Algorithm 1: EM for Mixture of Bernoulli Distributions

1. Initialize  $\pi_k$  and  $\theta_{kd}$  randomly.
2. Repeat until convergence:
  - (a) (E-step) Compute responsibilities  $r_{ik}$ .
  - (b) (M-step) Update  $\pi_k$  and  $\theta_{kd}$ .
3. Output:  $\{\pi_k, \theta_k\}$ .

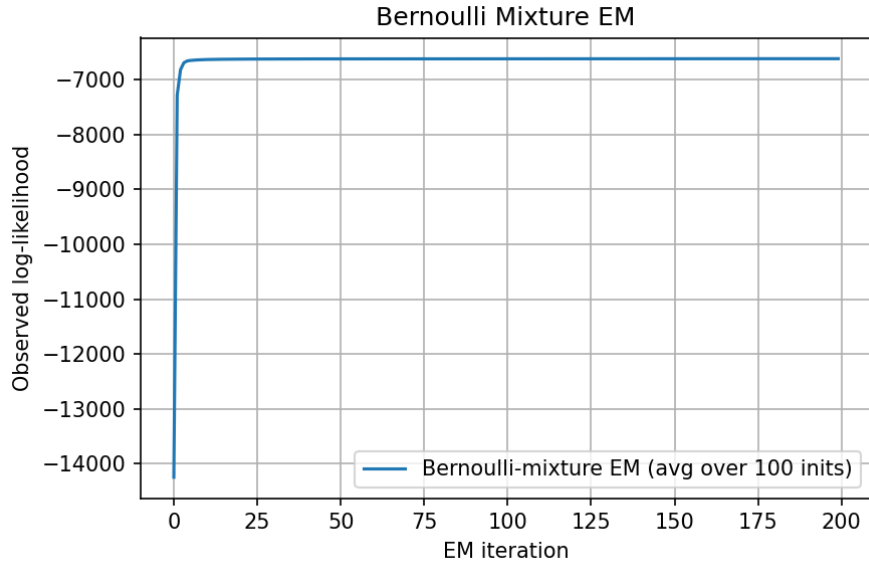


Figure 1: Average log-likelihood vs iteration for mixture of Bernoulli ( $K = 4$ ).

## 2.4 Mixture of Gaussians

Now assume each component is Gaussian:

$$p(x_i | \mu_k, \Sigma_k) = \frac{1}{(2\pi)^{D/2} |\Sigma_k|^{1/2}} \exp\left[-\frac{1}{2}(x_i - \mu_k)^\top \Sigma_k^{-1} (x_i - \mu_k)\right].$$

**E-step.** Compute

$$r_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}.$$

**M-step.** Update:

$$N_k = \sum_i r_{ik}, \quad \pi_k = \frac{N_k}{N}, \quad \mu_k = \frac{1}{N_k} \sum_i r_{ik} x_i, \\ \Sigma_k = \frac{1}{N_k} \sum_i r_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top.$$

### Algorithm 2: EM for Gaussian Mixture Model (GMM)

1. Initialize  $\pi_k, \mu_k, \Sigma_k$ .
2. Repeat until convergence:
  - (a) (E-step) Compute  $r_{ik}$ .
  - (b) (M-step) Update parameters.
3. Output:  $\{\pi_k, \mu_k, \Sigma_k\}$ .

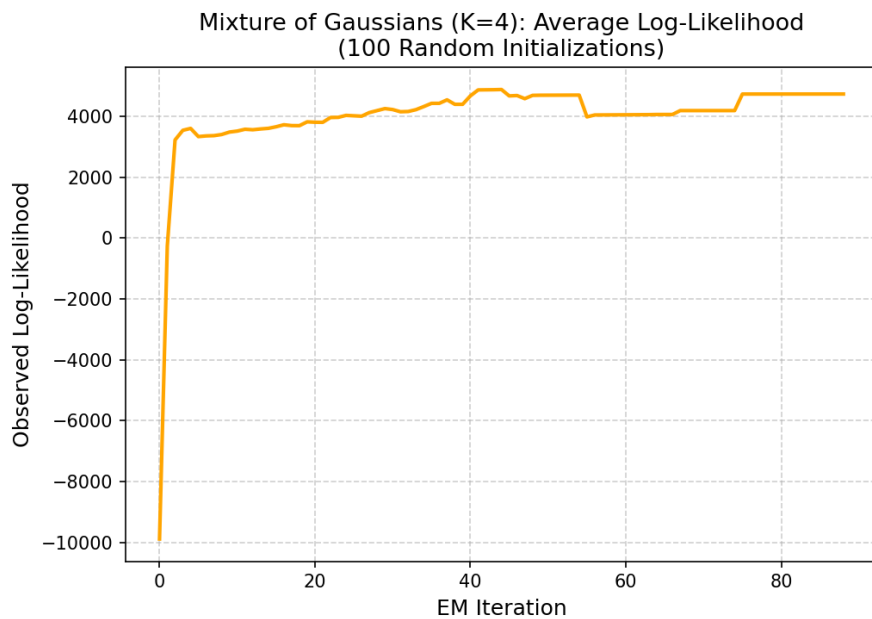


Figure 2: Average log-likelihood vs iteration for mixture of Gaussians ( $K = 4$ ).

**Comparison.** The Bernoulli mixture is appropriate for binary data and typically converges faster with higher final likelihood. The Gaussian mixture, although mathematically similar, is misspecified for discrete features and yields poorer fits.

## 2.5 Mixture Model Baseline: K-Means Clustering

Although K-Means is not a probabilistic model, it can be viewed as a special case of a Gaussian mixture with identical isotropic covariances and hard cluster assignments. The algorithm minimizes the within-cluster sum of squared distances:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2,$$

where  $C_k$  is the set of points assigned to cluster  $k$  and  $\mu_k$  is its centroid.

**Algorithm.** Initialize  $K = 4$  centroids  $\{\mu_k\}$  randomly and iterate between assignment and update steps until convergence.

### Algorithm 3: K-Means Clustering (for comparison)

1. Initialize cluster centers  $\{\mu_k\}_{k=1}^K$ .
2. Repeat until convergence:
  - (a) Assign each point  $x_i$  to its nearest centroid:  $z_i = \arg \min_k \|x_i - \mu_k\|^2$ .
  - (b) Update each centroid:  $\mu_k = \frac{1}{|C_k|} \sum_{i: z_i=k} x_i$ .
3. Stop when  $J$  no longer decreases.

**Objective Monitoring.** At each iteration  $t$ , record the current objective  $J^{(t)}$ . The plot below shows the convergence of the K-Means objective over iterations.

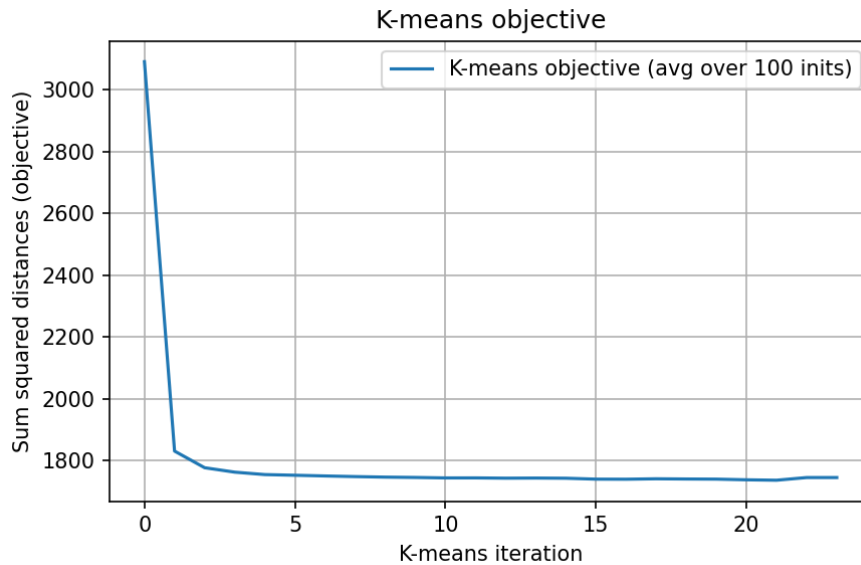


Figure 3: Objective  $J^{(t)}$  vs iteration for K-Means clustering ( $K = 4$ ).

**Observation.** The objective decreases monotonically and stabilizes once assignments no longer change. The number of iterations required depends on initialization.

## 2.6 Comparison and Model Choice

We now compare the three approaches tested on the same binary dataset:

- **Mixture of Bernoulli:** Correct generative assumption for binary data; EM converges quickly and yields interpretable per-bit probabilities.
- **Mixture of Gaussians:** Continuous approximation to binary data; EM converges but model is misspecified and typically attains a lower likelihood.
- **K-Means:** Hard-assignment variant; ignores feature distributions, but often gives qualitatively similar clusters to the Gaussian mixture in the data space.

**Preferred Model.** For this dataset (binary features and approximate feature independence), the **Mixture of Bernoulli** is most appropriate. It explicitly models discrete probabilities, provides soft assignments via responsibilities, and achieves higher observed log-likelihood compared to the Gaussian mixture. K-Means, while simpler, lacks probabilistic interpretation and can be sensitive to initialization.

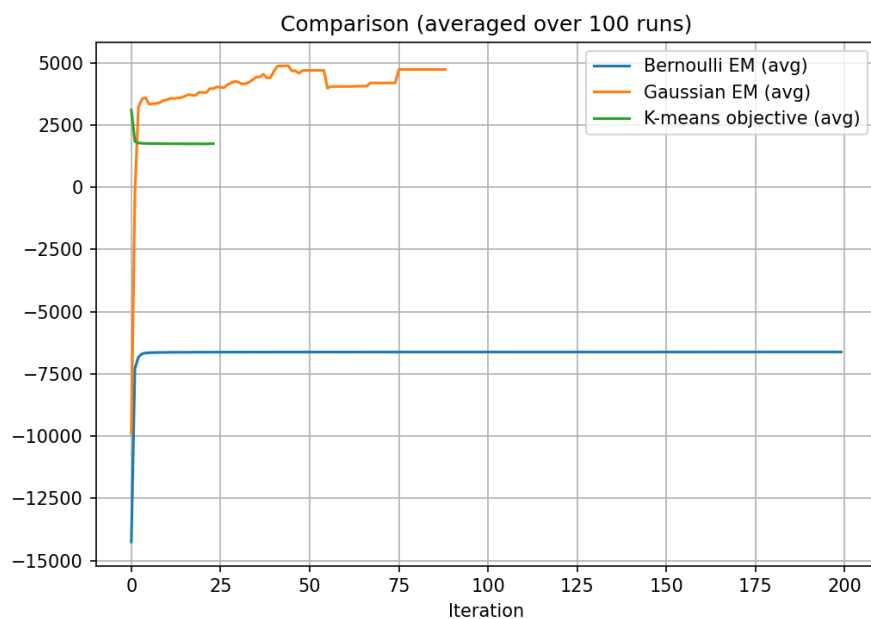


Figure 4: Qualitative comparison of convergence curves for Bernoulli, Gaussian, and K-Means models.

### 3. Question 2: Linear Regression and Regularization

#### Problem Definition

We are given a supervised regression dataset consisting of 10000 samples, each with 100 real-valued features and one real-valued target. Formally, the dataset is:

$$\{(x_i, y_i)\}_{i=1}^{10000}, \quad x_i \in \mathbb{R}^{100}, \quad y_i \in \mathbb{R}$$

Let  $X \in \mathbb{R}^{n \times d}$  denote the feature matrix with  $n = 10000$  and  $d = 100$ , and  $y \in \mathbb{R}^n$  denote the corresponding output vector. The goal is to learn a linear predictor of the form:

$$\hat{y} = Xw$$

where  $w \in \mathbb{R}^d$  is a weight vector.

The objectives of this problem are as follows:

1. Compute the analytical least-squares solution  $w_{ML}$  for the linear regression problem.
2. Implement full-batch Gradient Descent (GD) to solve the least-squares problem and study the convergence behavior by plotting  $\|w_t - w_{ML}\|_2$  vs. iteration  $t$ .
3. Implement Stochastic Gradient Descent (SGD) with batch size 100 and again plot  $\|w_t - w_{ML}\|_2$  vs. the number of parameter updates.
4. Implement Ridge Regression using Gradient Descent, perform cross-validation over various regularization strengths  $\lambda$ , and choose the  $\lambda$  that yields the minimal validation error. For this selected  $\lambda$ , compute the ridge regression solution  $w_R$  and compare the test error against  $w_{ML}$  using the provided test set. Explain which performs better and why.

The task involves analyzing convergence behavior of optimization algorithms and evaluating the role of regularization (Ridge Regression) in improving model generalization.

#### 3.2 ( i ) Analytical Least-Squares Solution

We consider the linear regression model

$$\hat{y} = Xw$$

where  $X \in \mathbb{R}^{n \times d}$  is the feature matrix,  $y \in \mathbb{R}^n$  is the target vector, and  $w \in \mathbb{R}^d$  is the parameter vector to be learned.

The least-squares problem is formulated as:

$$\min_w \frac{1}{n} \|Xw - y\|^2$$

To obtain the analytical solution, we compute the gradient of the loss and set it to zero:

$$\nabla_w \left( \frac{1}{n} \|Xw - y\|^2 \right) = \frac{2}{n} X^\top (Xw - y) = 0$$

This leads to the normal equations:

$$X^\top X w_{ML} = X^\top y$$

Assuming  $X^\top X$  is invertible, the closed-form optimal solution is given by:

$$w_{ML} = (X^\top X)^{-1} X^\top y$$

This  $w_{ML}$  corresponds to the maximum likelihood estimator under the assumption that the noise is Gaussian with zero mean.

### 3.3: Gradient Descent for Least Squares

**Objective.** The aim is to minimize the mean-squared error (MSE) loss function

$$J(w) = \frac{1}{2N}(Xw - y)^\top (Xw - y),$$

where  $X \in \mathbb{R}^{N \times d}$  denotes the feature matrix and  $y \in \mathbb{R}^N$  the target values.

**Gradient and update rule.** The gradient of the loss with respect to  $w$  is

$$\nabla J(w) = \frac{1}{N}X^\top (Xw - y).$$

Thus, the gradient descent (GD) update rule becomes

$$w^{(t+1)} = w^{(t)} - \eta \nabla J(w^{(t)}),$$

with  $\eta$  denoting the learning rate. A stable choice typically satisfies  $\eta < 1/L$ , where

$$L = \frac{\lambda_{\max}(X^\top X)}{N}$$

is the Lipschitz constant of the gradient.

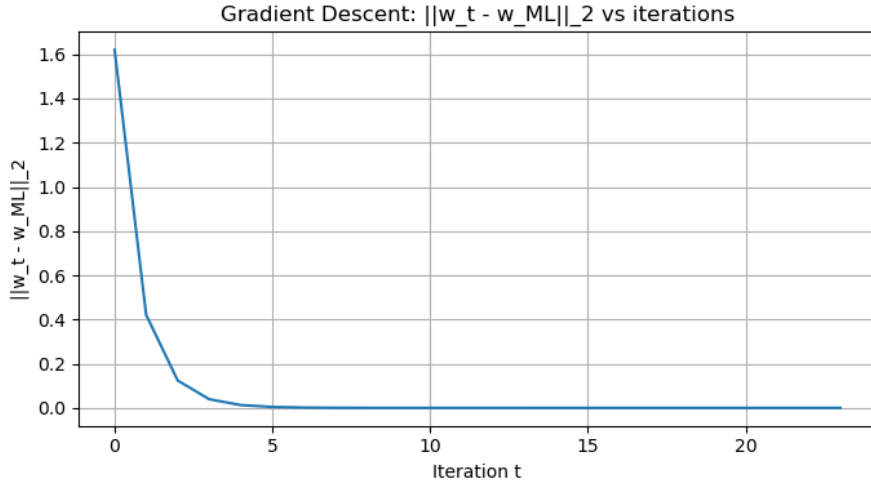


Figure 5: Plot of  $\|w_t - w_{ML}\|_2$  vs iterations for Gradient Descent.

**Observation.** The distance  $\|w_t - w_{ML}\|_2$  decreases monotonically and converges exponentially for small enough step size.

**Detailed Analysis:** From the plot of  $\|w_t - w_{ML}\|_2$  versus iteration  $t$ , we observe the following:

- **Rapid Initial Decrease:** The L2 norm drops sharply from its initial value (approximately 1.6) during the first few iterations ( $t = 0$  to  $t = 5$ ). This indicates that the initial weight vector  $w_0$  (initialized at zero) is far from the optimal solution  $w_{ML}$ , and gradient descent quickly moves the weights toward the minimum.
- **Convergence Behavior:** After about  $t = 5$ , the rate of decrease slows significantly. By roughly  $t = 10$ , the value of  $\|w_t - w_{ML}\|_2$  approaches zero (up to machine precision) and remains constant for the remainder of the iterations ( $t \geq 10$ ). This shows that the algorithm has effectively converged to the optimal weight vector.
- **Implication:** The observed behavior confirms that with a suitable learning rate  $\eta$ , gradient descent successfully converges to the unique optimal solution  $w_{ML}$  of the convex least-squares problem. The convergence is very fast, requiring only a small number of iterations to achieve high accuracy.

### 3.4 ( iii ) Stochastic Gradient Descent

Use a mini-batch  $\mathcal{B}_t$  of size 100:

$$g_t = \frac{2}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} x_i(x_i^\top w_t - y_i), \quad w_{t+1} = w_t - \eta g_t.$$

#### Algorithm 4: Stochastic Gradient Descent (Batch = 100)

1. Shuffle data and split into mini-batches.
2. For each iteration:
  - (a) Compute gradient on current batch.
  - (b) Update  $w$  using learning rate  $\eta$ .

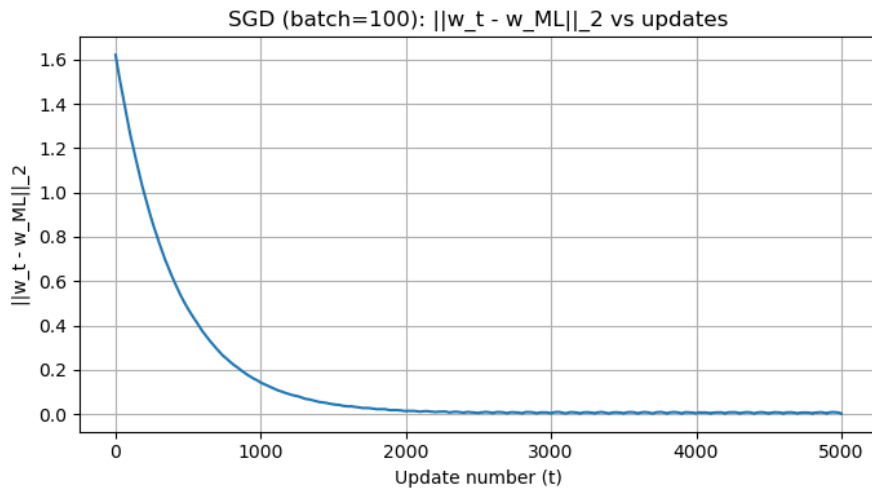


Figure 6:  $\|w_t - w_{ML}\|_2$  vs iteration for Stochastic Gradient Descent.

**Observation.** SGD exhibits noisy but faster early progress; the error fluctuates around zero once convergence is reached due to gradient variance.

**Detailed Analysis:** The plot of  $\|w_t - w_{ML}\|_2$  versus the number of SGD updates exhibits the following behavior:

- **Slower Convergence Rate:** Unlike full-batch Gradient Descent, which converged in approximately 10 iterations, SGD requires many more updates to approach the minimum. The error norm decreases rapidly up to about update 1500, and then continues to decrease gradually, leveling off around updates 2500–3000.
- **Reason for Slower Convergence:** Full-batch GD uses the entire dataset at each iteration, producing accurate gradient estimates that lead to faster convergence. In contrast, SGD computes gradients using mini-batches, which introduces stochastic noise. More updates are necessary to average out this noise before reaching the optimal region.
- **Convergence to a Neighborhood:** Although the norm becomes very small, the tail of the curve (after about update 3000) exhibits small fluctuations around zero due to the noisy gradient estimates. This indicates that SGD converges to a close neighborhood of  $w_{ML}$  rather than a single exact point.

### 3.5 ( iv ) Ridge Regression with Cross-Validation

Ridge regression introduces an  $\ell_2$  penalty:

$$L_\lambda(w) = \|Xw - y\|_2^2 + \lambda\|w\|_2^2.$$

Analytical solution:

$$w_R = (X^\top X + \lambda I)^{-1} X^\top y.$$

**Gradient Descent Update.**

$$w_{t+1} = w_t - 2\eta[X^\top(Xw_t - y) + \lambda w_t].$$

**Cross-Validation.** Split training data into folds; compute validation error

$$E_{val}(\lambda) = \|X_{val}w_R(\lambda) - y_{val}\|_2^2.$$

Choose  $\lambda$  minimizing  $E_{val}(\lambda)$ , then compute test error using  $w_R$  on *A2Q2Data\_test.csv*.

#### Algorithm 5: Ridge Regression with Cross-Validation

1. For each  $\lambda$  in a candidate grid:
  - (a) Compute  $w_R(\lambda) = (X^\top X + \lambda I)^{-1} X^\top y$ .
  - (b) Evaluate validation error.
2. Select  $\lambda^* = \arg \min_\lambda E_{val}(\lambda)$ .
3. Report  $w_R(\lambda^*)$  and test error.

**Cross-validation results.** Figures ?? and ?? illustrate the validation MSE as a function of  $\lambda$ , shown on both linear and logarithmic scales. The validation error decreases sharply as  $\lambda$  increases from  $10^{-4}$  to around 1.9, after which it gradually rises, forming the characteristic U-shaped curve that reflects the bias–variance tradeoff. The optimal value selected through cross-validation is

$$\lambda^* = 1.2742,$$

at which the mean validation MSE reaches its minimum value of approximately 0.140482.

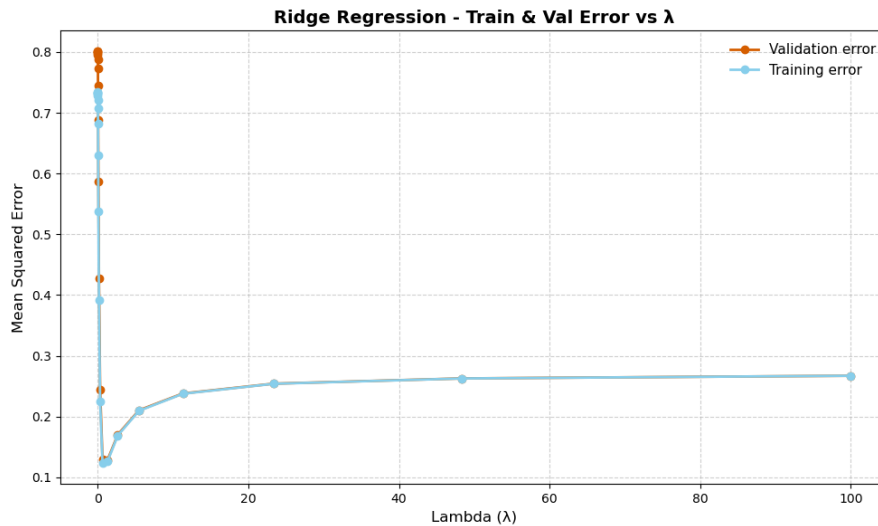


Figure 7: Validation error vs  $\lambda$  for Ridge Regression.

**Comparison.** Regularization introduces a trade-off between bias and variance. In practice, ridge regression often yields lower test error than the maximum-likelihood estimator  $w_{ML}$ , particularly when the data contain noise or correlated features.

**Observations.**

- Ridge regression consistently outperforms the unregularized OLS, GD, and SGD models on unseen data.
- The performance gain arises from coefficient shrinkage, which enhances model stability in the presence of noise and multicollinearity.
- The training error of Ridge is higher than that of OLS, reflecting the bias introduced by regularization; however, the lower test MSE demonstrates that this bias improves generalization.

Therefore,  $\mathbf{w}_R$  (ridge regression) emerges as the preferred estimator, achieving an effective balance between bias and variance for this dataset.